KIT-Fakultät für Physik
Institut für Experimentelle Teilchenphysik

Lectures: Prof. Dr. M. Klute, Dr. P. Goldenzweig
Exercises: Dr. P. Cheema, Dr. N. Trevisani
Assistants: S. Giappichini, J. Hornung, A. Monsch, D. Wong

Exercises for Particle Physics I

Winter Semester 2026/27

Exercise 3

To be worked on until [**TBD**] January 2027

In this exercise, we will go through several aspects of calorimetry in high energy physics. We will start with the basic concepts related to the interaction of particles with matter and how they are exploited to perform energy measurements. Then, we will use simulated events in a calorimeter with the geometry of the PHENIX electromagnetic calorimeter to visualize some examples of what we can expect to observe in a real experiment. Each particle typically deposits energy in several channels of a calorimeter, which introduces the problem of how to associate each energy deposit to the correct particle, and how to correctly count how many particles deposited energy in the calorimeter. Clustering algorithms are fundamental tools to fulfill this task: we will introduce the k-means algorithm, a relatively simple and intuitive algorithm to perform clustering. We will guide you through the implementation of the k-means algorithm, and you will compare your own creation with the implementation available in scikit-learn! We will then introduce a more realistic approach, based on the seeding of clusters. Finally, Section 4 connects k-means to the Expectation-Maximization algorithm and to Variational Autoencoders (see the pages after this one).

As you will quickly notice, the notebook has a lot of content: we know this. The hand-in and review will be in January 2027 (**exact date:** [**TBD**]), and we will offer one session where you can collect your questions and doubts in November 2026 (**date and time:** [**TBD**]).

To start with the exercise, please login to the jupytermachine, start the standard `Datenanalyse` container and update your `tp1_forstudents` repository, e.g. by navigating to the directory in the file browser on the left and then selecting `Git -> Pull from Remote` from the menu bar. A new subfolder `Exercise03` should appear with a Jupyter notebook `Exercise03.ipynb`. Open the notebook and work through the different tasks that are given inside.

Section 4: Latent-variable models — from k-means to EM to VAE

The last section of the notebook revisits the clustering problem from the point of view of *latent-variable models*, following the lecture "Variational Autoencoder" (K. He, MIT 6.S978 *Deep Generative Models*, 2024). You will see that the k-means algorithm of Section 3, the Expectation-Maximization (EM) algorithm and the Variational Autoencoder (VAE) are three ways of maximising one and the same objective, and you will use them for three calorimetry tasks: sharing the energy of overlapping showers, choosing the number of clusters, and learning what a "normal" electromagnetic shower looks like.

The generative picture

A latent (hidden) variable z is drawn from a prior $p(z)$ and a generator produces the observation, $x \sim p_\theta(x \,|\, z)$. In a calorimeter this is the physics itself: for a single *hit*, z says which particle produced it and $p_\theta(x|z)$ is the lateral shower profile around the impact point; for a whole *shower*, z contains the impact point, energy and shape, and $p_\theta(x|z)$ is the electromagnetic cascade plus the detector response. For any distribution $q(z)$ the log-likelihood can be decomposed as

$$\log p_\theta(x) \;=\; \underbrace{\mathbb{E}_{q(z)}\big[\log p_\theta(x \,|\, z)\big] - D_{\mathrm{KL}}\big(q(z) \,\|\, p(z)\big)}_{\mathrm{ELBO}(q,\theta)} \;+\; \underbrace{D_{\mathrm{KL}}\big(q(z) \,\|\, p_\theta(z \,|\, x)\big)}_{\geq 0}, \tag{1}$$

so the *evidence lower bound* (ELBO) is a lower bound on $\log p_\theta(x)$, which becomes tight when q equals the posterior $p_\theta(z|x)$. The three methods differ only in how they represent q and how they maximise the ELBO (table 1 and fig. 1).

	representation of $q(z)$	maximisation of the ELBO	latent z
k-means	one-hot (hard) assignment	alternate: assign / update	discrete
EM (Gaussian mixture)	exact posterior (soft)	alternate: E-step / M-step	discrete
VAE	encoder network $q_\phi(z \,|\, x)$	stochastic gradient descent	continuous

Table 1: Three ways to maximise the ELBO of eq. (1).

EM for an energy-weighted Gaussian mixture

Model the hit positions x_i of an event as a mixture of K Gaussians, $p_\theta(x) = \sum_k \pi_k \,\mathcal{N}(x \,|\, \mu_k, \Sigma_k)$, and give every hit a weight $w_i \propto E_i$ (normalised to $\sum_i w_i = N$). EM alternates

$$\text{E-step:}\quad r_{ik} = \frac{\pi_k \,\mathcal{N}(x_i|\mu_k,\Sigma_k)}{\sum_j \pi_j \,\mathcal{N}(x_i|\mu_j,\Sigma_j)}, \tag{2}$$

$$\text{M-step:}\quad N_k = \sum_i w_i r_{ik}, \qquad \pi_k = \frac{N_k}{\sum_i w_i}, \qquad \mu_k = \frac{1}{N_k}\sum_i w_i r_{ik}\, x_i,$$

$$\Sigma_k = \frac{1}{N_k}\sum_i w_i r_{ik}\,(x_i-\mu_k)(x_i-\mu_k)^{\mathsf{T}} + \varepsilon\,\mathbb{I}. \tag{3}$$

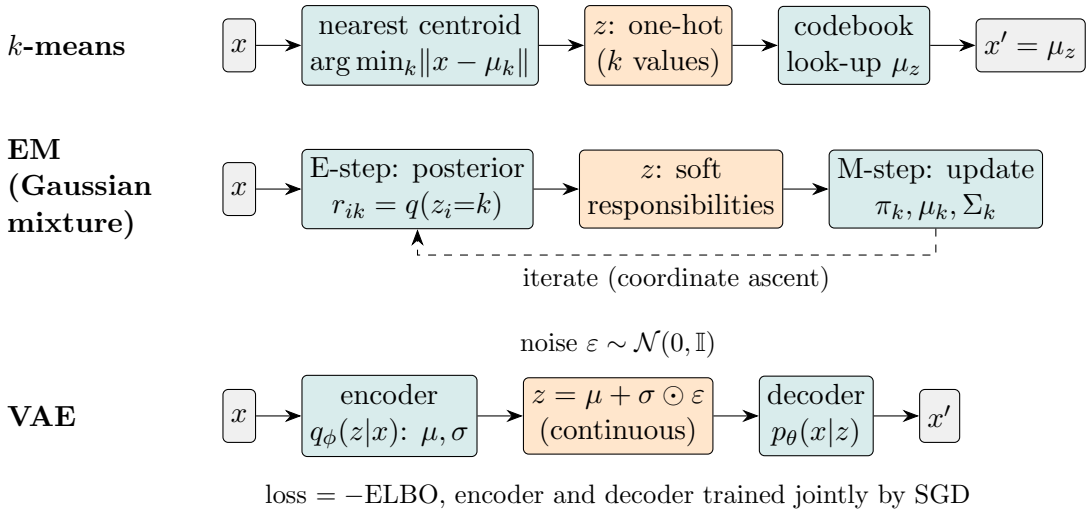


Figure 1: k-means is an autoencoder with a discrete bottleneck (top); EM replaces the hard assignment by the exact, soft posterior and alternates two maximisation steps (middle); the VAE replaces the analytic posterior by an encoder network and the discrete latent by a continuous one (bottom).

The term $\varepsilon = (5.5\,\mathrm{cm})^2/12$ appears because a hit is a *tower*, not a point: averaging $\log \mathcal{N}$ over the tower area adds $-\frac{\varepsilon}{2}\,\mathrm{tr}\,\Sigma_k^{-1}$ to the log-likelihood. Both steps increase the ELBO, so EM converges monotonically. With $\Sigma_k = \sigma^2 \mathbb{I}$ and $\sigma \to 0$ the responsibilities become one-hot and EM turns into (energy-weighted) k-means. The soft responsibilities share the energy of a tower between overlapping showers, $E_k = \sum_i r_{ik} E_i$ (fig. 2), and the likelihood provides a principled way to choose K, the Bayesian Information Criterion $\mathrm{BIC} = -2 \log p_{\hat{\theta}}(x) + n_{\mathrm{par}} \ln N$.

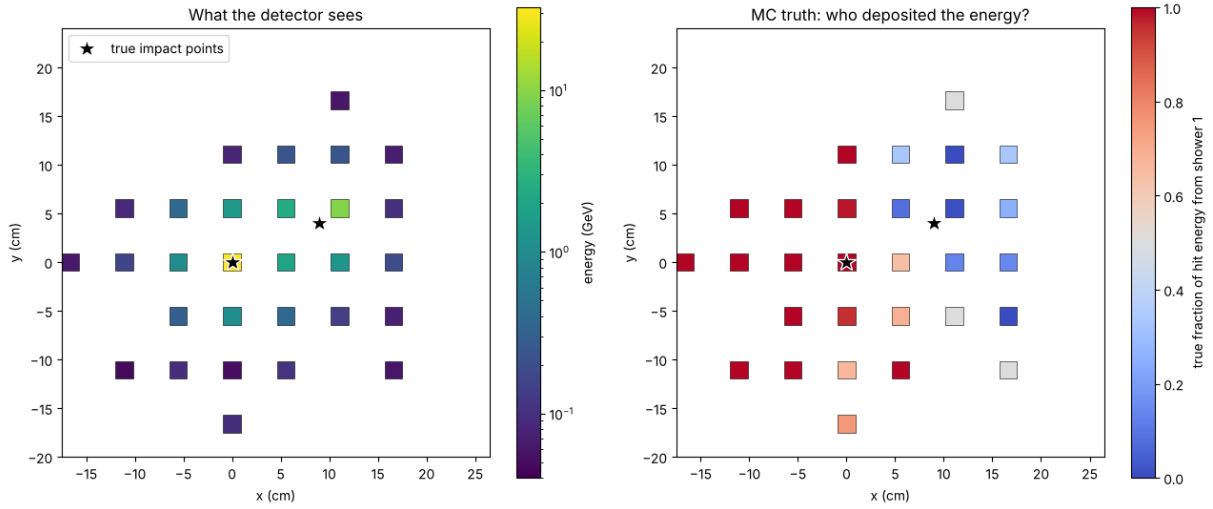


Figure 2: A toy event with two overlapping showers (40 and 15 GeV). Left: measured tower energies. Right: Monte-Carlo truth — the fraction of each tower's energy deposited by shower 1. Towers in between contain energy from both showers, which a hard assignment cannot describe.

A VAE for shower images

For Problems 4.3 and 4.4 a data point is a whole shower: the 7×7 tower energies around the seed, normalised to a sum of one (fig. 3). The VAE has a Gaussian encoder $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \operatorname{diag}\sigma^2_\phi(x))$, a prior $p(z) = \mathcal{N}(0, \mathbb{I})$ and a decoder that predicts the energy fraction $p_j(z)$ of every tower. Assuming the energy arrives in N_q independent "quanta" gives a multinomial likelihood, and the loss per shower is

$$\mathcal{L}(x) = -N_q \sum_j x_j \log p_j(z) \; + \; \tfrac{1}{2} \sum_l \left(\mu_l^2 + \sigma_l^2 - \log \sigma_l^2 - 1\right), \qquad z = \mu + \sigma \odot \varepsilon, \;\; \varepsilon \sim \mathcal{N}(0, \mathbb{I}). \quad (4)$$

The *reparametrisation* $z = \mu + \sigma \odot \varepsilon$ moves the randomness into an input, so that gradients can flow through the sampling step. The VAE is written in plain NumPy, so that every term of the loss and of its gradient is visible.

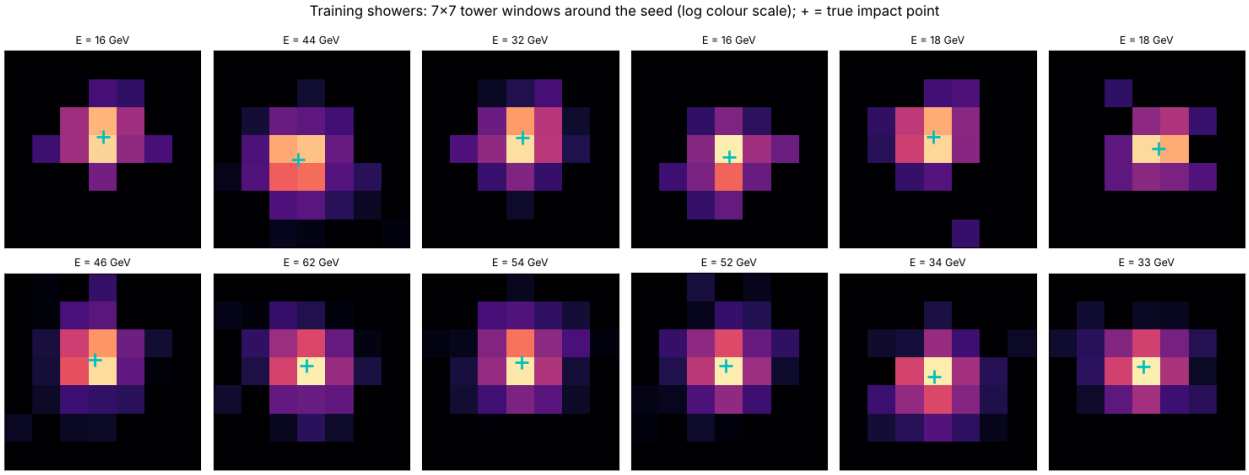


Figure 3: Training data for Problems 4.3 and 4.4: 7×7 tower windows around the seed tower of simulated electromagnetic showers (logarithmic colour scale); the cross marks the true impact point.

Your tasks in Section 4

The PHENIX samples contain no information about which particle produced which hit, and too few events to train a network. The notebook therefore provides a fast toy simulation with full Monte-Carlo truth, tuned to the shower containment of the PHENIX electrons: `ex3.generate_shower_event` (events with several showers) and `ex3.make_shower_images` (shower windows). Only four cells require your code (marked `# --> TODO`); the analysis and plotting cells are provided, so concentrate on interpreting the plots.

Problem 4.1 Energy-weighted k-means, seeded with your `seed_search`: compare the position resolution with the unweighted version and with the seed-tower centre.

Problem 4.2 Implement the E-step, M-step and ELBO of an energy-weighted Gaussian mixture (unit test provided). Visualise EM as coordinate ascent on the ELBO, show that k-means

is its $\sigma \to 0$ limit, study the energy sharing as a function of the shower separation, choose K with the BIC, and apply the method to real $A' \to e^+e^-$ events.

Problem 4.3 Use your k-means class on 49-dimensional shower vectors: interpret the codebook of shower templates, use it as an autoencoder and study how much information a code of $\log_2 K$ bits carries.

Problem 4.4 Implement the reparametrisation, the KL term and the reconstruction loss (with gradients) of a VAE, train it (about a minute on a CPU) and explore the latent space, the decoded shower manifold, the generation of new showers and the use of the ELBO to detect merged clusters.

Further reading

- D. P. Kingma, M. Welling, *Auto-Encoding Variational Bayes*, ICLR 2014, arXiv:1312.6114.
- R. M. Neal, G. E. Hinton, *A view of the EM algorithm that justifies incremental, sparse, and other variants*, in *Learning in Graphical Models*, Springer, 1998.
- T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning*, Springer (Sec. 8.5: the EM algorithm).
- A. van den Oord, O. Vinyals, K. Kavukcuoglu, *Neural Discrete Representation Learning* (VQ-VAE), NeurIPS 2017, arXiv:1711.00937.
- G. Grindhammer, S. Peters, *The parameterized simulation of electromagnetic showers in homogeneous and sampling calorimeters*, arXiv:hep-ex/0001020.
- E. Nalisnick et al., *Do Deep Generative Models Know What They Don't Know?*, ICLR 2019, arXiv:1810.09136.